Lecture 27: Machine Learning Continued

Heather Mattie, PhD

December 11, 2025

Recipe of the Day!

Muddy Buddies

Announcements

- Lab this week on machine learning
- Homework #5 due Friday, December 12th
- Project files due Thursday, December 18th by 12pm (noon)
- There will be lectures held next week
  - Clustering and PCA
  - Text mining
  - Next steps in data science
- No TF office hours next week – just Heather's
- No lab next week

Machine Learning

LASSO and Ridge Regression

Heather Mattie, PhD

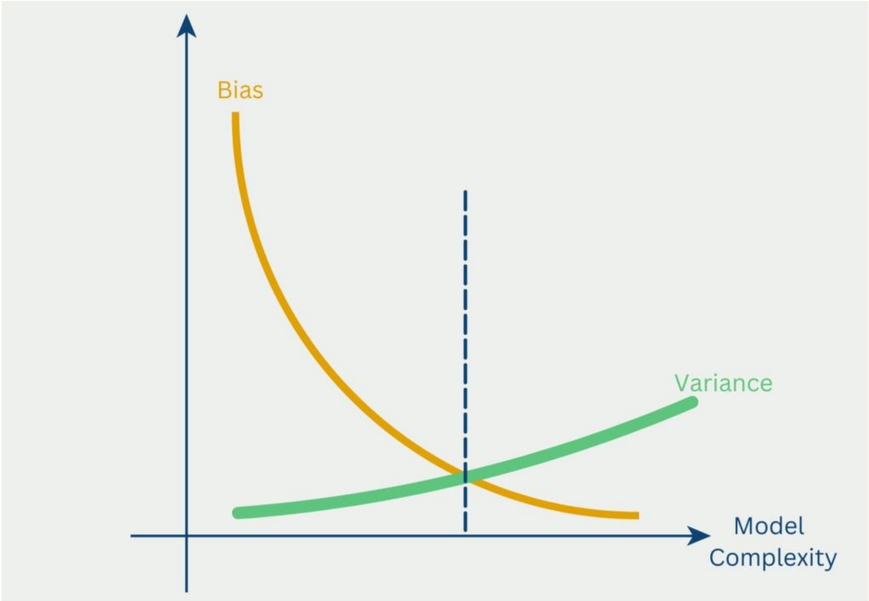
Types of errors

The prediction error for any machine learning model can be decomposed into three components: bias, variance, and irreducible error.

- Bias: the error that is introduced by approximating a real-life problem, which may be extremely complicated, by a much simpler model (too simple of a model).
- Variance: the amount by which the trained model would change if trained with a different training set.
- Irreducible error: no matter how good the model, the data will have a certain amount of noise or error that cannot be removed.

Bias-variance tradeoff

Bias and variance are related and a decrease in one will result in the increase of the other. The goal is to find a model with the least amount of bias and variance, keeping this tradeoff in mind.



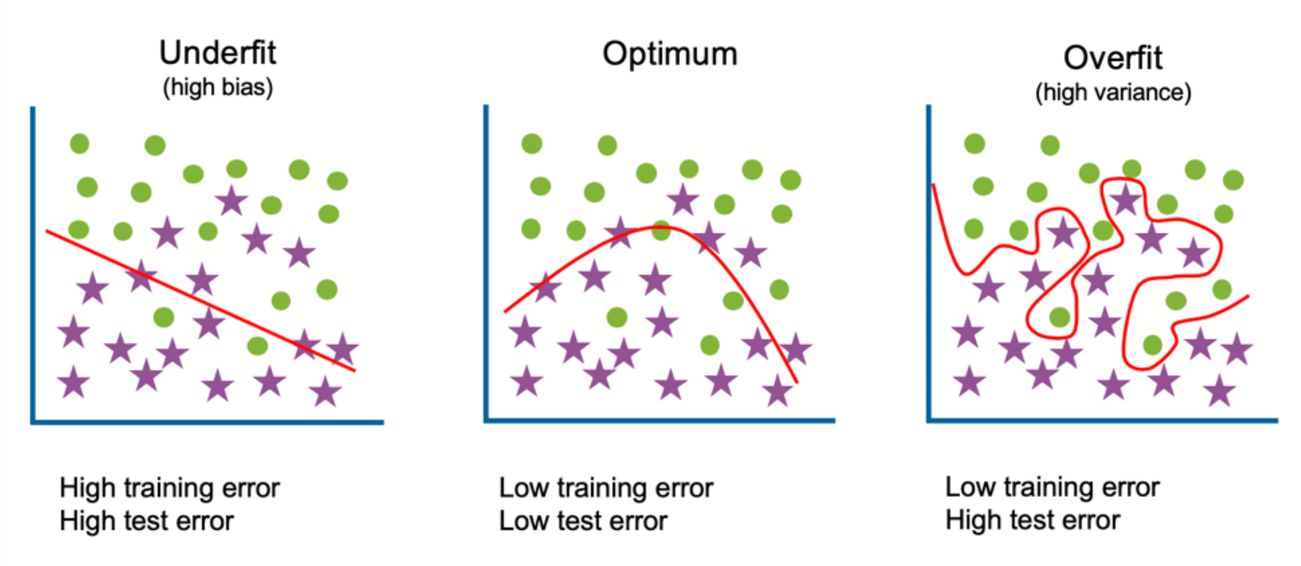
Total error = Variance + Bias squared + Irreducible Error

$$\mathbf{E}[y_0 - \hat{f}(x_0)]^2 = Var(\hat{f}(x_0)) + [Bias(\hat{f}(x_0))]^2 + Var(\epsilon)$$

Regularization

Regularization is a technique used in machine learning to prevent overfitting by decreasing variance. Two common methods:

- Shrinkage: LASSO and ridge regression
- Dimension reduction: principal component analysis (PCA)



LASSO

LASSO (least absolute shrinkage and selection operator) performs both regularization (preventing overfitting) and automatic feature selection by shrinking less important coefficients ***to* zero**.

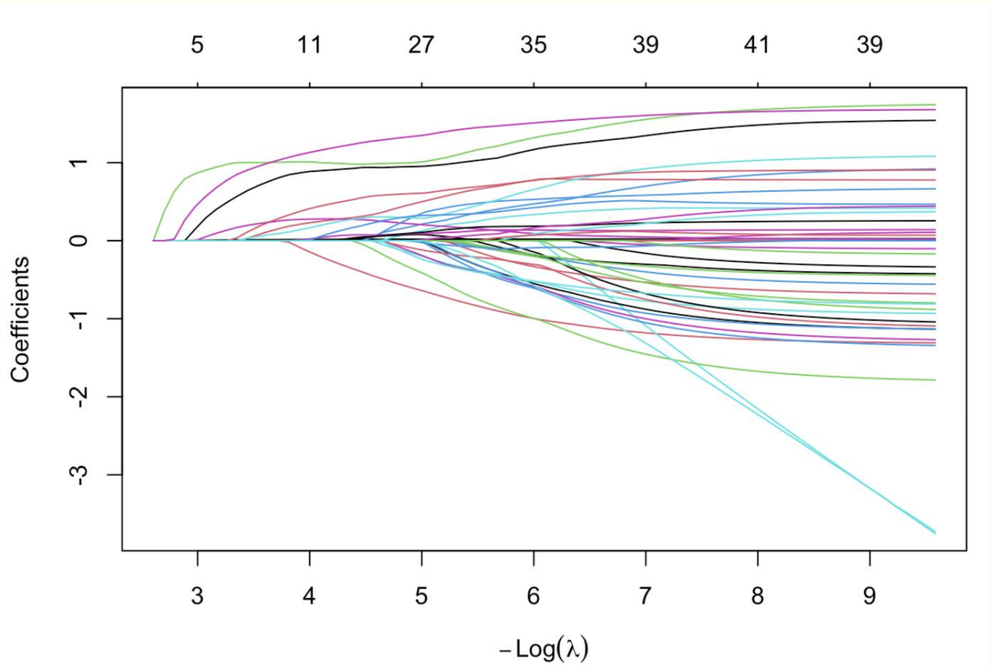
$$\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2 + \boxed{\lambda}\sum_{j=1}^{p}|\beta_j|$$

- Adds a penalty term λ (L1 norm) based on the absolute value of coefficients to the standard regression equation
- Effectively sets some coefficients to exactly zero

LASSO

- When $\lambda = 0$: standard linear (or logistic) regression since penalty term disappears. Low bias, but high variance.
- As λ increases, more and more coefficients are set to 0 since the goal is to minimize this equation. Bias is increased, but variance decreases at a faster rate.
- If λ is too large, bias increases more than variance decreases and the model is not complex enough (underfitting).

$$\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=1}^{p}|\beta_j|$$



Ridge Regression

Ridge regression performs both regularization (preventing overfitting) and automatic feature selection by shrinking less important coefficients ***toward* zero**.

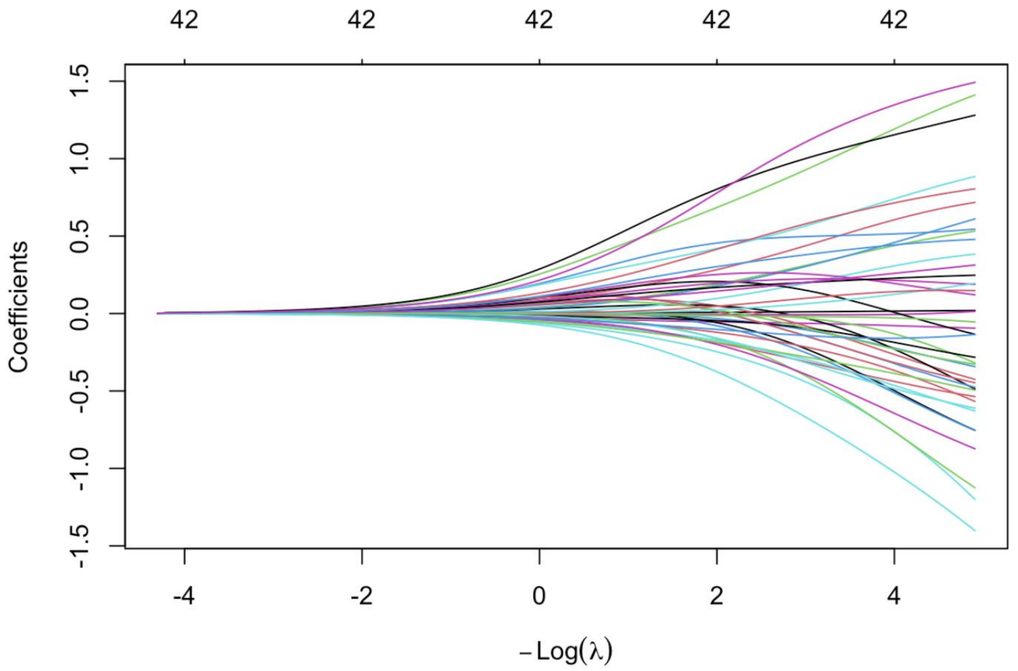
$$\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=1}^{p}\beta_j^2$$

- Adds a penalty term λ (L2 norm) based on the square value of coefficients to the standard regression equation
- Shrinks coefficients toward zero, but not to exactly zero

Ridge Regression

- When $\lambda = 0$: standard linear (or logistic) regression since penalty term disappears. Low bias, but high variance.
- As λ increases, more and more coefficients are shrunk towards 0 since the goal is to minimize this equation. Bias is increased, but variance decreases at a faster rate.
- If λ is too large, bias increases more than variance decreases and the model is not complex enough (underfitting).

$$\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=1}^{p}\beta_j^2$$



Finding the optimal λ

Finding the best value for λ can be done using k-fold cross-validation.

1. Choose a grid of λ values to try.
2. Randomly split the dataset into k non-overlapping groups or folds – common values for k are 5 and 10.
3. For each of the k folds:
   - Take the fold as the holdout or validation set.
   - Take the remaining k-1 folds as the training set.
   - Fit the model (lasso or ridge) on the training set for each value of λ.
   - Evaluate each of the models (trained with different values of λ on the validation set, and calculate performance, typically called the cross-validation error or loss.
4. Take the mean of the cross-validation errors across all k folds for each value of λ. The best tuning parameter is the one with the smallest mean CV error.

Example: MEPS hospitalization data

The Medical Expenditure Panel Survey (MEPS), conducted by the Agency for Healthcare Research and Quality, contains longitudinal information on participants' socio-demographics, health behaviors, existing conditions, access to care, health care utilization, and expenditures.

- Task: predict being hospitalized in 2017 based on data collected about the same individuals in 2016.
- Predictors (26 total) include age, sex, race, cancer, coronary heart disease, diabetes, perceived general and mental health status, any hospitalization in 2016, etc.

Example: MEPS hospitalization data

```
# Read in subset of MEPS data
load("meps.rData")

meps %>% dplyr::select(Age, Diabetes.diagnosis, Perceived.health, Anyhosp2016, Anyhosp2017) %>% head
```

```
##       Age Diabetes.diagnosis Perceived.health Anyhosp2016 Anyhosp2017
## 1017   39                 No        Excellent       FALSE       FALSE
## 8004   39                 No        Excellent       FALSE       FALSE
## 4775   34                 No        Very good       FALSE       FALSE
## 10369  44                 No        Very good       FALSE       FALSE
## 9725   30                 No        Excellent       FALSE       FALSE
## 10539  37                 No             Good       FALSE       FALSE
```

Example: MEPS hospitalization data

```
# Predictor variables
predictors = setdiff(names(meps), "Anyhosp2017")

# Create design matrix for predictors (except for intercept)
X = model.matrix(~ .-1, meps %>% dplyr::select(all_of(predictors)))

# Response variable
y = meps$Anyhosp2017

# Split the data into a training set and a test set
set.seed(219)
train_idx = createDataPartition(y, times = 1, p = 0.8, list = FALSE)
X_train = X[train_idx,]
y_train = y[train_idx]
X_test = X[-train_idx,]
y_test = y[-train_idx]
```

Preliminary models

The `glmnet` function from the `glmnet` package can be used to fit logistic regression, LASSO, and ridge regression models.

A logistic regression model will be trained as a baseline comparison of model performance.

```
fit_logistic = glmnet(X_train, y_train, family = "binomial", lambda = 0)
```

Preliminary models

The `glmnet` function from the `glmnet` package can be used to fit logistic regression, LASSO, and ridge regression models.

A logistic regression model will be trained as a baseline comparison of model performance.

- `family = "binomial"` indicates the outcome is binary
- `lambda = 0` indicates logistic regression

```
fit_logistic = glmnet(X_train, y_train, family = "binomial", lambda = 0)
```

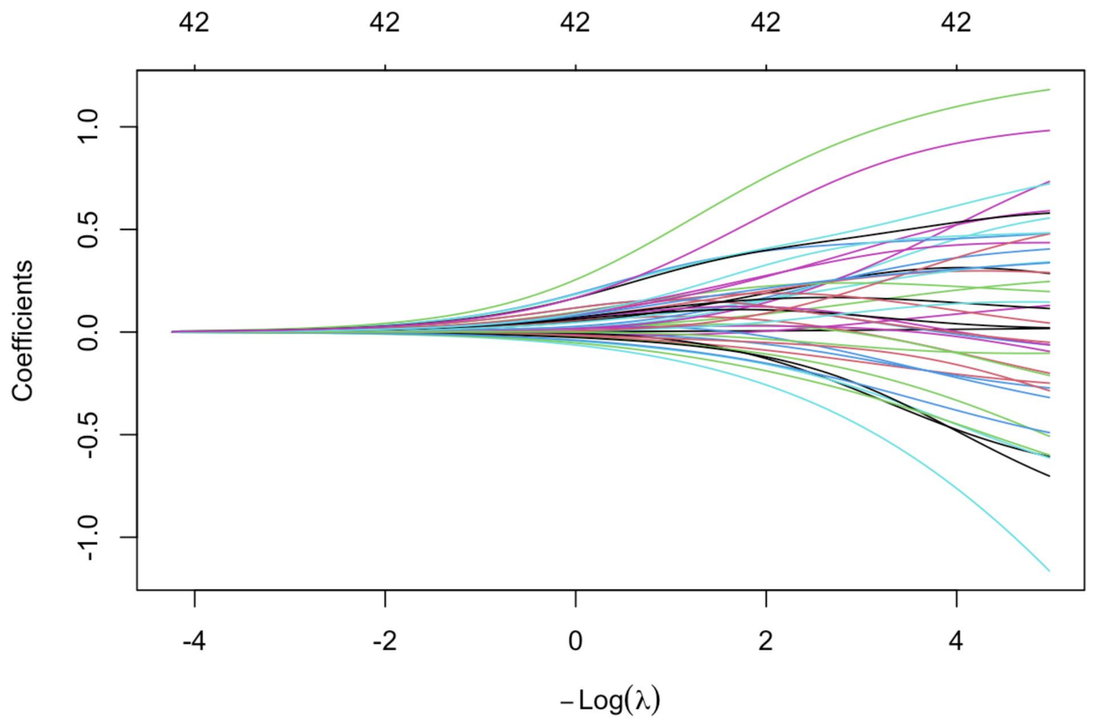
Preliminary models

- `alpha = 0` indicates ridge regression
- `alpha = 1` indicates LASSO
- If λ is not specified, `glmnet` will generate a grid of 100 values to try

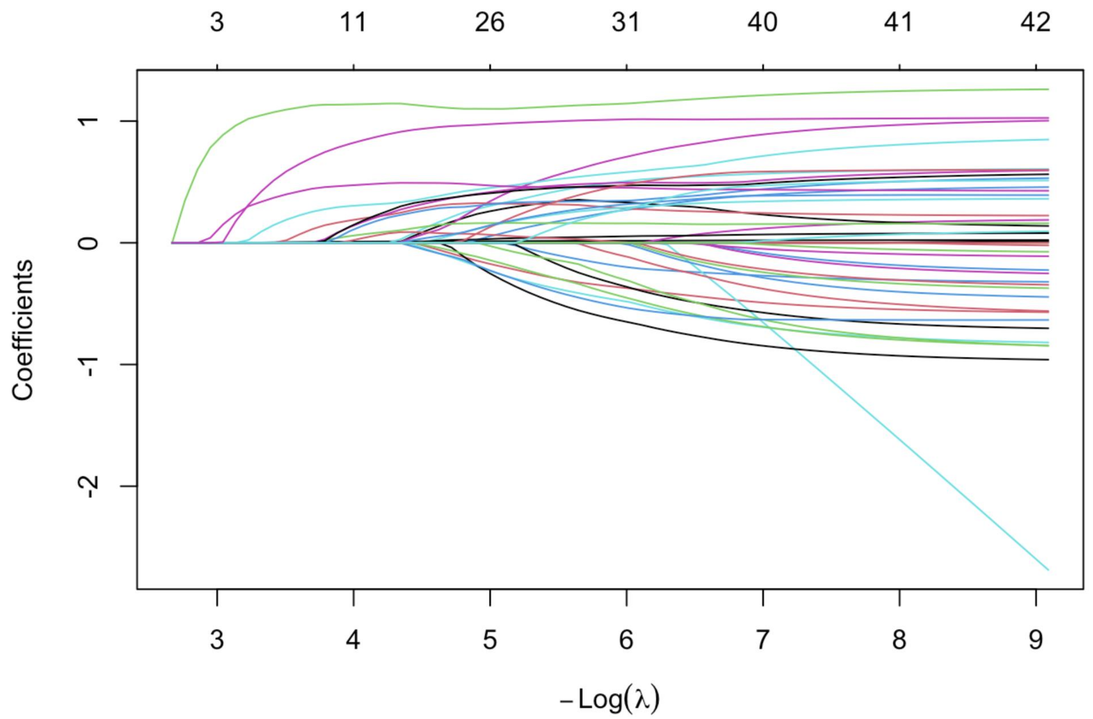
```
fit_ridge = glmnet(X_train, y_train, family = "binomial", alpha = 0)
fit_lasso = glmnet(X_train, y_train, family = "binomial", alpha = 1)
```

Coefficient values

```
plot(fit_ridge, xvar = "lambda")
```




```
plot(fit_lasso, xvar = "lambda")
```

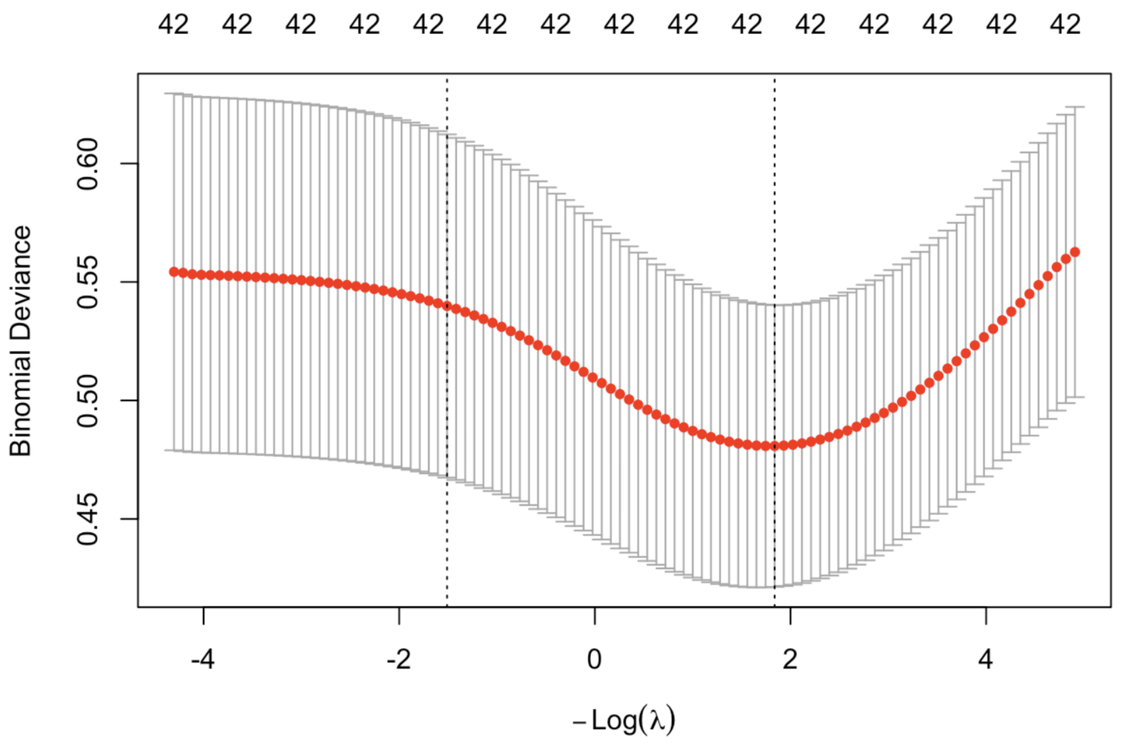


Cross-validation for λ

`cv.glmnet` is a built-in function that will run cross-validation to identify the best tuning parameter for the model.

- Default is 10-fold cross-validation
- Deviance (a type of error) is the default performance metric for quantifying cross-validation error

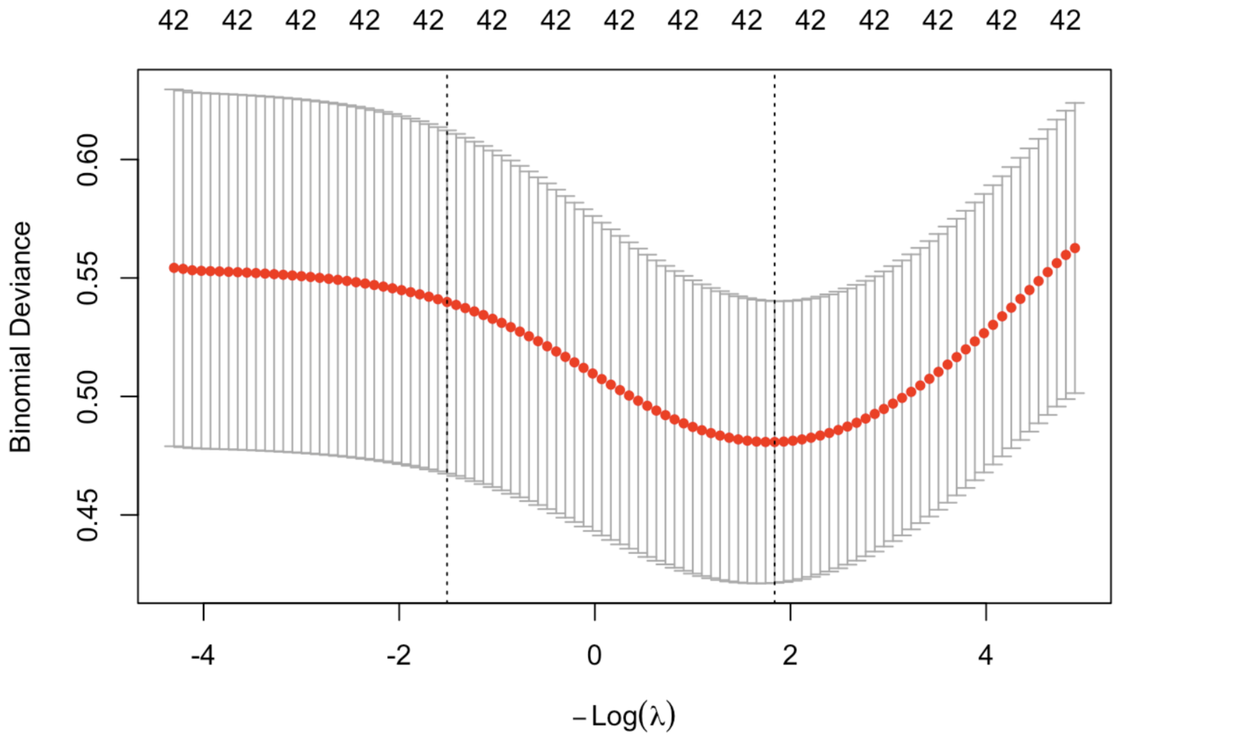
```
cv_ridge = cv.glmnet(X_train, y_train, family = "binomial", alpha = 0)
plot(cv_ridge)
```



Cross-validation for λ

- The dashed line on the right indicates a λ value of that is one standard deviation above the value that minimizes the cross-validation error (dashed line on the left).
- This is named **`lambda.1se`** and is sometimes preferred as a stronger penalty that further reduces the risk of overfitting.
- The value of λ that minimizes CV error is named **`lambda.min`**.

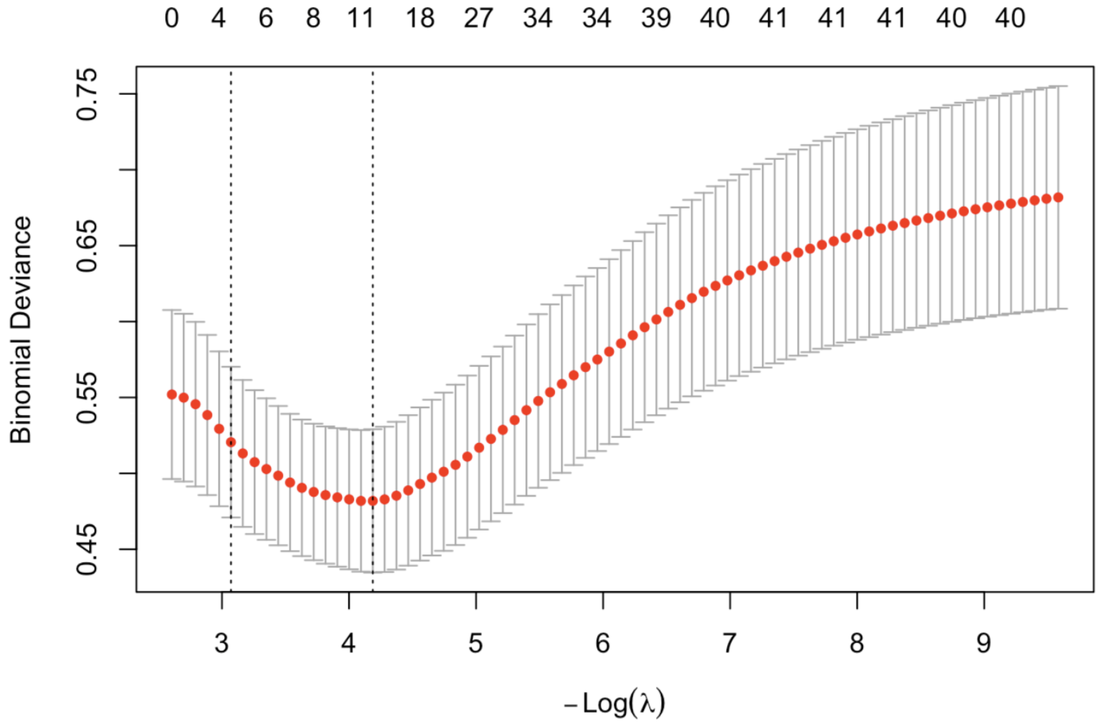
```
cv_ridge = cv.glmnet(X_train, y_train, family = "binomial", alpha = 0)
plot(cv_ridge)
```



Cross-validation for λ

- The dashed line on the right indicates a λ value of that is one standard deviation above the value that minimizes the cross-validation error (dashed line on the left).
- This is named **`lambda.1se`** and is sometimes preferred as a stronger penalty that further reduces the risk of overfitting.
- The value of λ that minimizes CV error is named **`lambda.min`**.

```
cv_lasso = cv.glmnet(X_train, y_train, family = "binomial", alpha = 1)
plot(cv_lasso)
```



Shrinkage

Ridge regression shrunk 2.27% of the predictors to 0 (25 predictors remain)

```
ridge_zero_coef = cbind(
  lambda = c(cv_ridge$lambda.min, cv_ridge$lambda.1se),
  "% 0 coef" = c(mean(coef(cv_ridge, s = "lambda.min") == 0),
                 mean(coef(cv_ridge) == 0)) * 100)
rownames(ridge_zero_coef) = c("min", "1se")
ridge_zero_coef
```

```
##         lambda % 0 coef
## min 0.1491276 2.272727
## 1se 1.2672170 2.272727
```

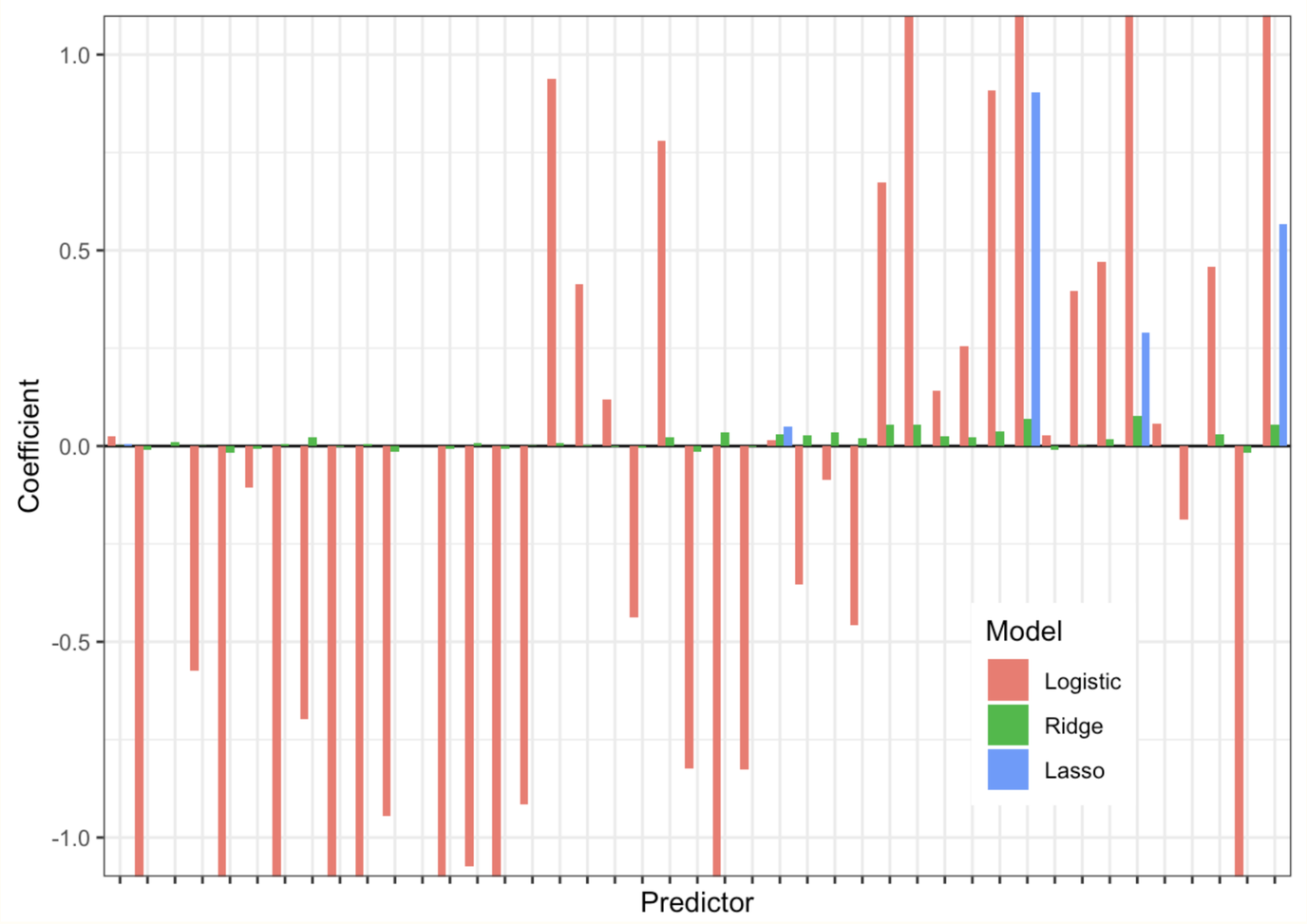
LASSO shrunk 72.7% of the predictors to 0 (only 6 predictors remain)

```
lasso_zero_coef = cbind(
  lambda = c(cv_lasso$lambda.min, cv_lasso$lambda.1se),
  "% 0 coef" = c(mean(coef(cv_lasso, s="lambda.min") == 0),
                 mean(coef(cv_lasso) == 0)) * 100)

rownames(lasso_zero_coef) = c("min", "1se")
lasso_zero_coef
```

```
##          lambda % 0 coef
## min 0.01714605 72.72727
## 1se 0.06921890 97.72727
```

Coefficient values



Model performance: logistic regression

```
p_hat_logit <- as.vector(predict(fit_logistic, newx = X_test, type = "response"))

y_hat_logit <- ifelse(p_hat_logit >= 0.5, 1, 0)

y_test <- as.factor(ifelse(y_test == "FALSE", 0, 1))

confusionMatrix(data = as.factor(y_hat_logit),
                reference = y_test, positive = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 181  13
##          1   3   2
##
##                Accuracy : 0.9196
```

Model performance: LASSO

```r
# Predicted probabilities from CV lasso model at lambda.min
p_hat_lasso <- as.vector(
  predict(cv_lasso,
          newx = X_test,
          s = "lambda.min",
          type = "response"))

y_hat_lasso <- factor(ifelse(p_hat_lasso >= 0.5, 1, 0))

confusionMatrix(data      = y_hat_lasso,
                reference = y_test,
                positive  = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 183  15
##          1   1   0
##
##                Accuracy : 0.9196
```

Model performance: ridge regression

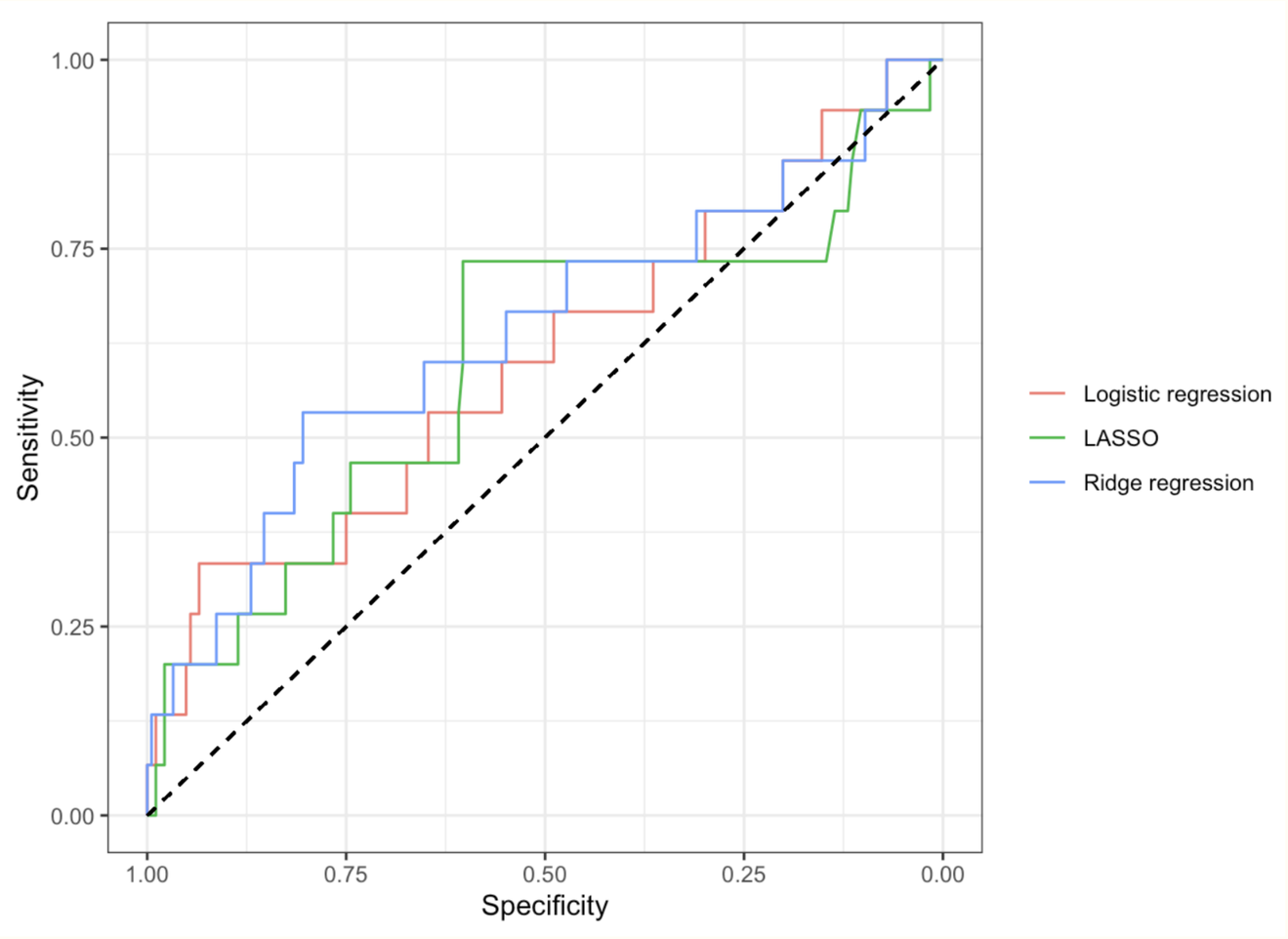
```r
# Predicted probabilities from CV ridge model at lambda.min
p_hat_ridge <- as.vector(
  predict(cv_ridge,
          newx = X_test,
          s = "lambda.min",
          type = "response"))

y_hat_ridge <- factor(ifelse(p_hat_ridge >= 0.5, 1, 0))

confusionMatrix(data      = y_hat_ridge,
                reference = y_test,
                positive  = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 184  14
##          1   0   1
##
##                Accuracy : 0.9296
```

Model performance




```
auc(roc_logitistic)
```

```
## Area under the curve: 0.6014
```

```
auc(roc_lasso)
```

```
## Area under the curve: 0.5982
```

```
auc(roc_ridge)
```

```
## Area under the curve: 0.638
```

Model performance

Model	Overall Accuracy	Sensitivity	Specificity	AUROC
Logistic regression	0.9196	0.1333	0.9837	0.6014
LASSO	0.9196	0	0.99457	0.5982
Ridge regression	0.9296	0.06667	1	0.638

None of the models are great and the next step would be trying other machine learning models like Random Forest.

Summary

LASSO Strengths

- Performs automatic variable selection by shrinking some coefficients exactly to zero
- Produces simpler, more interpretable models
- Helps reduce overfitting when many predictors are irrelevant
- Works well when only a small subset of predictors truly matter

LASSO Weaknesses

- When predictors are highly correlated, LASSO tends to select one and ignore the others
- Can be unstable in situations with multicollinearity
- Struggles when p >> n and predictors are highly correlated
- May overshrink coefficients, reducing predictive performance in some settings

Summary

Ridge Regression Strengths

- Handles multicollinearity very well by shrinking correlated predictors together
- Keeps all predictors in the model, useful when they all carry some signal
- Often provides better predictive performance than LASSO when many small/weak effects exist
- More stable than LASSO (less sensitive to small data changes)

Ridge Regression Weaknesses

- Does not perform variable selection - coefficients never shrink to zero
- Resulting models can be harder to interpret
- May keep irrelevant predictors in the model
- Not ideal when the goal is a sparse, easy-to-interpret model